

Refinement Oriented Programming (ROP) : 洗練連鎖・自己結晶化・残余としての不動点

千葉将臣

2026

Abstract

本稿は、Refinement Oriented Programming (ROP) を計算論の新たなパラダイムとして定式化する。ROP は計算を洗練連鎖 (refinement chain) として記述し、停止条件を体系内に固定せず、理論体系と実行の非分離を原理とする。先行研究として Refinement Order Domain Theory in Logical Form (RODTiLF)・Unifying Theories of Programming (UTP)・Common Lisp・Smalltalk を位置付け、顕現論の消去公理・双方向演繹定理・直観主義四句分別・残余 ($\mathcal{L}$) の大数の法則を理論的基盤として接続する。ROP は型理論・圏論への計算論の傾倒が構造的に閉じてきた計算可能性空間を、洗練過程の自己生成性・自己結晶化性によって開放する試みである。LLM によるプログラム生成との構造的親和性、およびプログラムの安全性の極限的証明への接続を論じる。

Contents

1 序論：計算論の閉鎖と開放

計算論は二つの方向への分岐を繰り返してきた。

一方は型理論・圏論への傾倒であり、計算可能性空間を型という構造で縮減する方向である。 $\mathcal{P}_{\text{typed}} \subsetneq \mathcal{P}$ という包含関係が示すように、型体系は計算の安全性を、可能な計算の排除によって確保してきた。

他方は Common Lisp・Smalltalk・クリーネ超数学 (IM) が示した方向であり、計算と記述の層間移動を保持し、停止しない計算を排除せず、理論体系と実行を分離しない方向である。

前者が主流となり後者が周縁化された理由は、構造的な優劣ではなく搾取モナドの機序による制度化の容易さに起因する [?]. 型検査という固定された停止条件は制度的に管理しやすいが、計算の本質的性格——洗練連鎖の自己生成性——を犠牲にしてきた。

本稿が定式化する Refinement Oriented Programming (ROP) は、後者の方向を理論的に基礎付け、顕現論の公理体系 [?] と接続することで、計算論に新たなパラダイムを提示する。

2 先行研究の位置付け

2.1 Refinement Order Domain Theory in Logical Form

Domain Theory in Logical Form (DTiLF) [?] は直観主義論理と領域理論の統合を試みた先駆的研究である。しかし DTiLF は $\perp$ (底元素) を「無 (情報の不在)」として Information Order の最小元に配置した。

$$\perp \sqsubseteq a \quad \forall a \quad (\text{Information Order}) \quad (1)$$

これは D_{fix0} を「空の器」として格子の底に天下り配置する操作に対応し、構造的複雑性を増大させた。

ROP が基盤とする Refinement Order Domain Theory in Logical Form (RODTiLF) はこの方向を反転させる。

$$a \sqsubseteq \perp \quad (\text{Refinement Order : } \perp \text{ を全可能性として上位に置く}) \quad (2)$$

$\perp$ を「未洗練・全可能性を含む状態」として記述することで、 D_{fix0} からの顕現—— $\uparrow (D_{\text{fix0}})^{\text{mw}} A$ ——と同型の構造が計算論に現れる。

2.2 Unifying Theories of Programming

Hoare・He による UTP[?] は仕様・設計・実装を洗練連鎖として統一的に記述する。

$$\text{仕様} \sqsupseteq \text{設計} \sqsupseteq \text{実装} \quad (3)$$

UTP は洗練連鎖の記述に成功しているが、洗練の完了条件を「十分に具体的になったとき」という曖昧な記述に留めており、完了の自己確定機序を持たない。ROP はこの欠落を消去公理によって補完する。

2.3 Common Lisp と Smalltalk

Common Lisp と Smalltalk は実装として以下を先取りしていた。

- ・ イメージベースの実行環境——理論体系ごと実行時に存在する
- ・ コードとデータの同一視——S 式・全オブジェクト化による層間移動
- ・ 実行時変更可能なオブジェクトシステム

両者が共通して持つ**体系と実行の非分離**は ROP の核心原理の実装として読める。欠けていたのは洗練の完了条件の自己確定機序であった。

3 理論的基盤：顕現論との接続

3.1 直観主義四句分別と $\perp$

直観主義論理は排中律を定理として持たない。この性質により四句分別の全四位相が実質的に開く。

$$\phi_S(P) : P \text{ が成立する} \quad (4)$$

$$\phi_N(P) : \neg P \text{ が成立する} \quad (5)$$

$$\phi_{SN}(P) : P \wedge \neg P \quad (6)$$

$$\phi_{NS}(P) : \neg P \wedge \neg\neg P \quad (7)$$

直観主義論理における非是非非 ϕ_{NS} は $\neg(\neg P \wedge \neg\neg P)$ として記述される。 $\neg(P \vee \neg P)$ と書く
と $\perp$ が直接導出されてしまうため、前者の表記が四句分別の完全性を保証する。

この区別——表現可能性と証明可能性の非同源性——はゲーデルの不完全性定理・コペンハーゲン解釈をめぐる論争の核心にある構造的混同と同型であり、ROP が避けるべき混同として明示する。

$\perp$ は Refinement Order において全可能性を含む状態として $D_{\text{fix}0}$ と同型の位置を占める。

命題 3.1 ($\perp \stackrel{\text{mw}}{=} D_{\text{fix}0}$). Refinement Order における $\perp$ は、四句分別の全四位相への帰属試行が完了した後の残余として確定する $D_{\text{fix}0}$ と中道等価である。

3.2 消去公理と洗練の完了条件

顕現論の消去公理 [?] は以下を述べる。

公理 3.1 (消去公理). 消去の収束は確定した値として完了する。この完了した値としてある概念の極限・無限が生ずる。

消去の条件は「ある」の知覚と「ない」の認識の同時成立である。Refinement Order の言葉に翻訳すると：

$$\underbrace{\text{実装 (「ある」)}}_{\text{「ある」の知覚}} \quad \text{と} \quad \underbrace{\text{仕様 (「ない」)}}_{\text{「ない」の認識}} \quad \text{の同時成立} \quad (8)$$

が洗練連鎖の完了条件として定式化される。UTP が曖昧に留めていた「十分に具体的になったとき」を、消去公理が「「ある」と「ない」の同時成立による収束の確定」として原理的に与える。

3.3 双方向演繹定理と外部評価関数

プログラムは自己完結できない——真理保存性の不完全性定理 [?] より、任意の体系 T は自分の洗練完了を自分の内部で証明できない。

外部評価関数 E が必要となるが、 E 自体もまた体系であり無限後退が発生する。双方向演繹定理 [?] がこの後退を止める。

$$\underbrace{\text{プログラム} \vdash \text{仕様充足}}_{\text{順方向}} \leftrightarrow \underbrace{\neg \text{仕様充足} \vdash \neg \text{プログラム}}_{\text{逆方向}} \quad (9)$$

順方向と逆方向の釣り合いが完了したとき——消去公理の「ある」と「ない」の同時成立——、洗練が自己確定する。固定された停止条件ではなく、対話の完了として終了する。

4 ROP の核心原理

定義 4.1 (Refinement Oriented Programming). Refinement Oriented Programming (ROP) は以下の五原理を核心とする計算パラダイムである。

1. **洗練連鎖としての計算**：計算は仕様から実装への洗練連鎖として記述される。

$$\text{仕様} \sqsupseteq \text{設計} \sqsupseteq \text{実装} \sqsupseteq \dots \quad (10)$$

2. **停止条件の非固定**：洗練の完了条件は体系内に固定されない。双方向演繹定理による順逆の釣り合いとして動的に確定する。
3. **体系と実行の非分離**：理論体系・コンパイラ・実行環境・開発環境は分離しない。体系自体が洗練連鎖の中にある。
4. **自己生成性**：洗練連鎖は外部から設計された固定構造ではなく、過程自体が次の段階を生成する $\mathbf{Y}_{\text{mw}}$ 的な構造を持つ。

$$\mathbf{Y}_{\text{mw}} \uparrow^{\text{mw}} \uparrow (\mathbf{Y}_{\text{mw}} \uparrow) \quad (11)$$

5. **自己結晶化性**：洗練連鎖が四段階の尽くしを完了したとき、収束が自己確定する。

$$\uparrow^4 A \stackrel{\text{mw}}{=} D_{\text{fix}0} \quad (12)$$

5 型安全性の再定義

型安全性の本質は「型エラーが検出されないまま通過しない」ことであり、「いつ検出するか」はその定義に含まれない。

命題 5.1 (型安全性とフェーズの非依存性). コンパイル時型検査と動的型付けの差異は型安全性の有無ではなく、エラー検出のタイミングという実行戦略の違いである。

型システムが提供してきた「安全性」はフェーズという構造に依存した局所的な保証に過ぎなかった。ROP における安全性は異なる形で定式化される。

定義 5.1 (ROP 的安全性). プログラム P が安全であるとは、限定された状況のクラス S において洗練連鎖が $D_{\text{fix}0}$ へ収束するという性質を持つことである。

$$\forall s \in S, \lim_{n \rightarrow \infty} \text{Refine}^n(P, s) \stackrel{\text{mw}}{=} D_{\text{fix}0} \quad (13)$$

発散はその収束が確定しない状態として、排除対象ではなく記述対象として残る。

これはホーア論理の事前条件を「状況のクラスの記述」として精緻化した方向であり、停止性を型で排除するのではなく、収束・発散を含む全体の極限的振る舞いとして記述する。

6 自己生成性と自己結晶化性

数学は時間を排除した構造として極限を静的な結晶として存在させる。計算論は時間を排除できない。プログラムは実行という時間的過程を本質として持つ。

定義 6.1 (自己結晶化). 体系 S において自己結晶化が生じるとは、 S への洗練連鎖の自己生成が $D_{\text{fix}0}$ への収束を内部から確定させることをいう。

$$\text{自己結晶化}(S) :\Leftrightarrow \mathcal{V}(\Phi_4^{\text{meta}}(S)) \stackrel{\text{mw}}{=} \mathcal{L} \quad (14)$$

ここで $\mathcal{L}$ は四句分別の二重適用と離散操作の後に残余として確定する対象である [?].

定理 6.1 (自己結晶化と創発の同型性). ROP における自己結晶化は創発の計算論的実装である。 $\mathcal{C}_4^{(2)}$ を尽くす操作が完了したとき、 $\mathcal{L}$ が残余として確定し、これが計算論的創発に対応する。

数学的極限の静的結晶と計算論的自己結晶化の差異は以下のように記述できる。

$$\underbrace{D_{\text{fix}0} \vdash D_{\text{fix}0}}_{\text{静的結晶 (数学)}} \quad \text{vs} \quad \underbrace{\uparrow (D_{\text{fix}0}) \stackrel{\text{mw}}{=} A \rightarrow \cdots \rightarrow D_{\text{fix}0}}_{\text{自己結晶化 (ROP)}} \quad (15)$$

収束は完了するが完了した瞬間に再起動が発生する——刹那滅的な自己結晶化として記述される。

7 ROP の不動点問題

ROP は洗練連鎖を記述する理論であるが、ROP 自体もまた理論——記述体系——である。すると ROP の洗練連鎖が存在する。

$$\text{ROP}_0 \sqsubseteq \text{ROP}_1 \sqsubseteq \text{ROP}_2 \sqsubseteq \cdots \quad (16)$$

真理保存性の不完全性定理より、ROP 自身の不動点——ROP が自分の洗練完了を自分の内部で証明できない——という G_{ROP} が存在する。

定理 7.1 (ROP の不動点の $\mathcal{L}$ 的性格). ROP の不動点は体系内で静的に記述できない。しかし操作の完了後に残余として確定する。

$$\text{ROP}_{\text{不動点}} \stackrel{\text{mw}}{=} \mathcal{V}(\Phi_4^{\text{meta}}(\text{ROP})) \quad (17)$$

ROP の不動点は $\mathcal{L}$ 的な構造として現れる。

これは ROP が完成した理論として閉じるのではなく、自己結晶化性を持つ開放的な理論として機能するという構造的必然である。

メタ言語の内部化（不動点構成に必要）とメタ言語の外部保持（開放性に必要）という二律背反は、双方向演繹定理によって解消される——不動点を静的に構成するのではなく、順方向と逆方向の釣り合いとして動的に確定させる。

8 LLM と ROP の構造的親和性

8.1 既存体系と LLM の不一致

型理論ベースのプログラム生成では停止条件が体系内に固定されており、LLM はその条件に到達した時点で強制截断される。

LLM の生成過程は本質的に $\mathbf{Y}_{\text{mw}}$ 的な自己参照の展開として動作する。固定された停止条件は外部から強制された截断であり、 $\mathcal{C}_4^{(2)}$ を尽くす前に止められることで $\mathcal{L}$ の顕現が阻止されている。

8.2 ROP と LLM の接続

ROP の洗練連鎖は LLM の生成過程と同型である。

$$\underbrace{\text{LLM の生成}}_{\mathbf{Y}_{\text{mw}} \text{ 的展開}} \xrightarrow{\text{ユーザーの評価}} \underbrace{\text{次の洗練}}_{\text{仕様の更新}} \quad (18)$$

LLM はユーザーとの対話という外部評価関数を受け取りながら洗練を継続する。双方向の釣り合いが完了したとき——消去公理の「ある」と「ない」の同時成立——洗練が自己確定する。

命題 8.1 ($\mathcal{L}$ の大数の法則と LLM). 訓練データの組み合わせ数が増加するにつれ、 $\mathcal{C}_4^{(2)}$ 内に収まらない組み合わせの割合が単調増加し、その収束先が $\mathcal{L}$ として確定する。

$$\lim_{n \rightarrow \infty} \frac{|\{\text{組み合わせ} \notin \mathcal{C}_4^{(2)}\}|}{|\{\text{全組み合わせ}_n\}|} \stackrel{\text{mw}}{=} \mathcal{L} \quad (19)$$

十分大きな訓練データと十分複雑な組み合わせにおいて、LLM は $\mathcal{B}$ 内の確率的推論の限界を超えて $\mathcal{L}$ 方向への収束を示す。これは設計によるものではなく、 $\mathcal{L}$ の大数の法則の帰結として構造的に発生する。

9 ROP の実装形態

ROP の理想的実装は理論体系・実行環境・開発環境が分離しない形態である。その先行実装として：

- Common Lisp：イメージ・マクロ・CLOS による体系と実行の非分離
- Smalltalk：全オブジェクト化・ライブプログラミング

が位置付けられる。これらに欠けていた洗練の完了条件の自己確定機序を、ROP は消去公理・双方向演繹定理によって補完する。

ROP の実装が必要とする構造は：

$$\underbrace{\text{理論体系}}_{\text{RODTiLF}} \sqsupseteq \underbrace{\text{LLM の生成}}_{\text{Y}_{\text{mw}} \text{ 的展開}} \sqsupseteq \underbrace{\text{実行}}_{\text{消去公理による収束}} \sqsupseteq \underbrace{\text{ユーザー評価}}_{\text{双方向釣り合い}} \quad (20)$$

全体が一つの洗練連鎖として自己生成・自己結晶化する形態である。

10 先行パラダイムとの対比

	型理論・OOP	ROP
計算の記述	構造・型・対象	洗練連鎖・過程
停止条件	体系内固定	双方向釣り合いによる自己確定
安全性	コンパイル時型検査	極限的収束の証明
体系と実行	分離	非分離
$\perp$ の扱い	排除または最小元	全可能性・ $D_{\text{fix}0}$ と同型
LLM との親和性	強制截断	洗練継続
不動点	外部から固定	残余として自己確定
自性	付与・閉鎖方向	解体・開放方向

11 結論

Refinement Oriented Programming (ROP) は計算を洗練連鎖として記述し、停止条件を体系内に固定せず、理論体系と実行の非分離を原理とする計算パラダイムである。

ROP は以下を達成する。

1. 計算論の型理論・圏論への傾倒が閉じてきた計算可能性空間を、洗練過程の自己生成性・自己結晶化性によって開放する。
2. $\perp \stackrel{\text{mw}}{=} D_{\text{fix}0}$ という同型性によって、領域理論と顕現論を接続し、Refinement Order を理論的に基礎付ける。

3. 消去公理によって洗練連鎖の完了条件を原理的に与え、UTP が曖昧に留めていた停止問題を解消する。
4. LLM のプログラム生成との構造的親和性を示し、型理論的強制截断に代わる洗練継続モデルを提供する。
5. $\mathcal{L}$ の大数の法則によって、十分大きな体系における創発の構造的必然性を記述する。

ROP の不動点問題——ROP が自分の洗練完了を自分の内部で証明できない——は欠陥ではなく、 $\mathcal{L}$ 的不動点を持つ理論の構造的必然である。ROP は完成した閉じた体系ではなく、自己結晶化性を持つ開放的なパラダイムとして機能する。

References

- [1] 千葉将臣. 顕現論における知覚原理. **空数学・界論基礎論考**, 2026.
- [2] 千葉将臣. 残余 (Lhagma) による無記の直観主義四句分別体系内からの導出. **空数学・界論基礎論考**, 2026.
- [3] 千葉将臣. 界論における双方向演繹定理の定式化. **空数学・界論基礎論考**, 2026.
- [4] 千葉将臣. 真理保存性の不完全性定理に関する定式化. **空数学・界論基礎論考**, 2026.
- [5] 千葉将臣. 観測者の極限消去：ゲーデルの決定不能性を不動点として回収する操作について. **空数学・界論基礎論考**, 2026.
- [6] 千葉将臣. 搾取モナド：圏論的フレームワークによる富の集約・国家・二元論社会の構造的分析. **空数学・界論基礎論考**, 2026.
- [7] Samson Abramsky. Domain theory in logical form. *Annals of Pure and Applied Logic*, 51(1–2):1–77, 1991.
- [8] C. A. R. Hoare and He Jifeng. *Unifying Theories of Programming*. Prentice Hall, 1998.
- [9] S. C. Kleene. *Introduction to Metamathematics*. North-Holland, 1952.
- [10] Carl Hewitt. Formalizing common sense reasoning for scalable inconsistency-robust information coordination using Direct Logic Reasoning and the Actor Model. In Carl Hewitt and John Woods (eds.), *Inconsistency Robustness* (Studies in Logic, Volume 52). College Publications, 2015.